

welcome to

FROM CODE TO PRODUCTION: ACCELERATING ENTERPRISE AI WITH MODERN TOOLS

poll



ABOUT ME

PhD from King's College London in Medical Imaging Sciences in 2015

Over a decade of experience designing and implementing ML/AI solutions using MLOPs-first approach

PyData Norwich Co-organizer



Gabriel Harris Ph.D.
Principal Data Scientist @ NIQ | Designing
and Implementing ML/AI solutions in Azure





**ESTABLISH
PROCESSES & BEST PRACTICES**



Python HOW: Using Poetry, Make, and pre-commit-hooks to Setup a Repo Template for your Team

Bring consistency, rigour, and best practices to your messy data science team

 Medium / Nov 15, 2022

*PyData Cambridge
Meetup 2023*

welcome to

**SETTING UP A REPOSITORY TEMPLATE
FOR YOUR TEAM**

with Consistency, Reproducibility, and Compliance in Mind

*PyData London
Conference 2023*

welcome to

**BRING BEST PRACTICES TO YOUR
MESSY DATA SCIENCE TEAM!**

with Reproducibility, Compliance, and Consistency in Mind

WHY?

—_ (ツ) _ /—
IT WORKS
on my machine

REPRODUCIBILITY

IT'S LIKE SOMEONE TOOK A
TRANSCRIPT OF A COUPLE
ARGUING AT IKEA AND MADE
RANDOM EDITS UNTIL IT
COMPILED WITHOUT ERRORS.



CONVENTIONS

EVENTUAL CONSISTENCY IS
GUARANTEED BY THE 2ND
LAW OF THERMODYNAMICS.
SOONER OR LATER
THIS WILL ALL BE A
UNIFORM HEAT BATH.
MAXIMUM
ENTROPY.



CONSISTENCY

HOW?



REPRODUCIBILITY



CONVENTIONS



CONSISTENCY

HOW?



REPRODUCIBILITY



CONVENTIONS



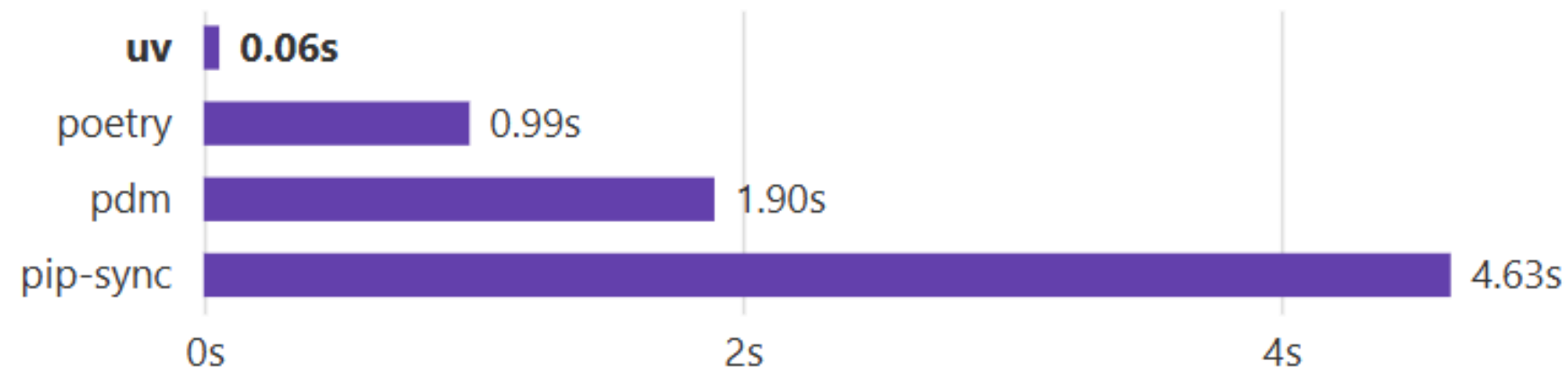
CONSISTENCY



85.9K

Stars

"An extremely fast Python package and project manager, written in **Rust**. Replaces pip, pip-tools, pipx, poetry, pyenv, virtualenv, and more."



Installing [Trio](#)'s dependencies with a warm cache.

astral-sh/uv

An extremely fast Python package and project manager, written in Rust.

559

Contributors

7

Used by

86k

Stars

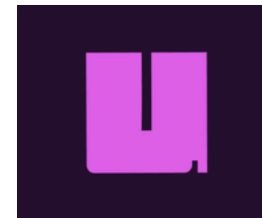
3k

Forks

astral-sh/uv: An extremely fast Python package and project manager, written in Rust.

An extremely fast Python package and project manager, written in Rust. - astral-sh/uv

GitHub



Install Python 3.12 from the the Astral python-build-standalone project

CLI

```
uv python install 3.12
```

```
uv init --python 3.12
```

```
uv venv
```

Create a virtual environment at .venv

Initialize a project, which creates:
pyproject.toml (contains project's metadata)

```
[project]
name = "pydata-norwich-sep-2025"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.12"
dependencies = []
```

.python-version (contains project's default
Python version that tells uv should use when
creating virtual environment)

```
3.12
```

PYPROJECT.TOML



"the purpose of [PEP 518](#) was to come up with a way for projects to specify what build tools they required ... all projects should (eventually) have ... development tools realized they ... could put their own configuration ... people liked the idea of centralizing configuration data in a single file"

– Brett Cannon, Python Steering Council

pyproject.toml

```
[project]
name = "pydata-norwich-sep-2025"
version = "0.1.0"
description = "Talk for Pydata Norwich Sep 2025"
readme = "README.md"
requires-python = ">=3.12"
dependencies = []
```


PYPROJECT.TOML

pyproject.toml

```
[project]
name = "pydata-norwich-sep-2025"
version = "0.1.0"
description = "Talk for Pydata Norwich Sep 2025"
readme = "README.md"
requires-python = ">=3.12"
dependencies = []

[tool.ruff]
line-length = 88
indent-width = 4
```

STRUCTURE

Three possible TOML tables:

- The [build-system] table ... allows you to declare which build backend you use ... to build your project
- The [project] table ... to specify your project's basic metadata
- The [tool] table has tool-specific subtables, ... contents are defined by each tool

Writing pyproject.toml



+ PYPROJECT.TOML

pyproject.toml

```
[project]
name = "pydata-norwich-sep-2025"
version = "0.1.0"
description = "Talk for Pydata Norwich Sep 2025"
readme = "README.md"
requires-python = ">=3.12"
dependencies = []

[dependency-groups]

dev = ["ruff", "pre-commit"]
features = ["polars", "seaborn"]
training = ["spacy>=3"]
inference = ["pydantic>=2"]
```

ISOLAT DEPENDANCY

Update .venv with different options

CLI

```
uv sync --all-groups
uv sync --only-dev
uv sync --only-group inference
```

Export lockfile to another format

CLI

```
uv export --all-groups --output-file requirements.txt
uv export --only-group inference --output-file inference.txt
```

breathe

HOW?



REPRODUCIBILITY



CONVENTIONS



CONSISTENCY



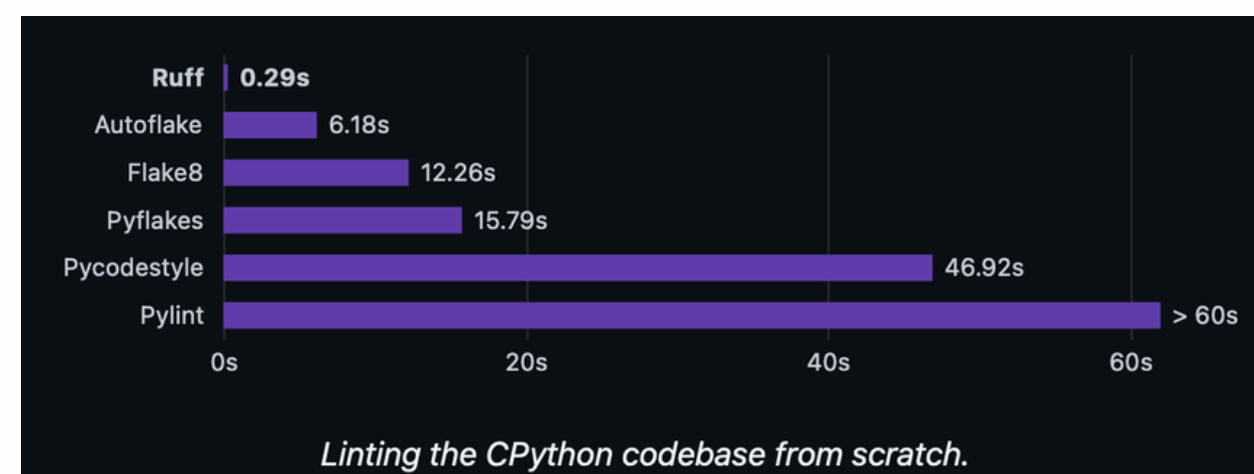
47.8K

Stars

"An extremely fast Python **linter** and **formatter**, written in **Rust**. Replaces Flake8, isort, pydocstyle (plus dozens of plugins)."

astral-sh/**ruff**

An extremely fast Python linter and code formatter, written in Rust.





INSTALL

OPTION 1

CLI



```
uv add ruff --dev
```

pyproject.toml



```
[dependency-groups]
dev = [
    "ruff>=0.13.2",
]
```

OPTION 2

pyproject.toml



```
[dependency-groups]
dev = [
    "ruff",
]
```

CLI



```
uv sync --only-dev
```

creates/updates

uv.lock



```
[package.metadata.requires-dev]
dev = [{ name = "ruff" }]

[[package]]
name = "ruff"
version = "0.13.2"
source = { registry = "https://pypi.org/simple" }
```

a cross-platform lockfile that contains exact information about dependencies. It should be checked into version control, allowing for consistent and reproducible installations across machines



LINTER

pyproject.toml

```
[tool.ruff.lint]
select = [
    "E",
    "F",
    "D",
    "I",
]
ignore = [
    "D100", "D101", "D102",
    "D103", "D104", "D105",
    "D106", "D107", "D206"
]

[tool.ruff.lint.pydocstyle]
convention = "google"
```

DEFINE

- E (**pycodestyle**): checks code against some of the style conventions in PEP 8
- F (**pyflakes**) checks code for errors
- D(**pydocstyle**): check compliance with most docstring conventions in PEP 257
- I (**isort**) sorts imports

ignore specific rules for pyflakes

settings for pydocstyle

CLI

```
uv run ruff check
uv run ruff check --fix
uv run ruff check path/to/code/
```

RUN

all files in the current directory (dry)
all files in the current directory (in-place)
a single file (in-place)

Ruff FORMATTER

pyproject.toml

```
● ● ●  
  
[tool.ruff]  
line-length = 88  
indent-width = 4  
  
[tool.ruff.format]  
quote-style = "double"  
indent-style = "space"  
docstring-code-format = false
```

DEFINE

Drop-in replacement for Black (and beyond)
configuration

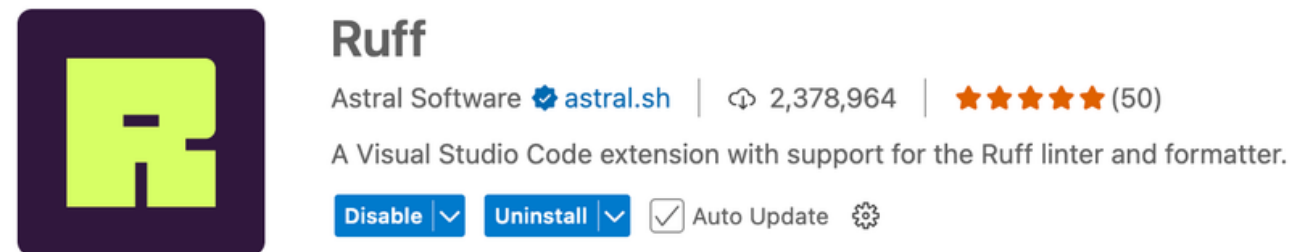
CLI

```
● ● ●  
  
uv run ruff format  
uv run ruff format path/to/code/  
  
uv run ruff format path/to/file.py  
uv run ruff format --check /path/to/file.py
```

RUN

all files in the current directory (in-place)
all files in `path/to/code` (in-place)
a single file (in-place)
a single file (dry)

Ruff **VSCODE INTEGRATION**



.vscode/settings.json

```
{
  "ruff.nativeServer": "on",
  "[python]": {
    "editor.formatOnSave": true,
    "editor.codeActionsOnSave": {
      "source.fixAll": "explicit",
      "source.organizeImports.ruff": "explicit"
    },
    "editor.defaultFormatter": "charliermarsh.ruff"
  },
  "notebook.formatOnSave.enabled": true,
  "notebook.codeActionsOnSave": {
    "notebook.source.fixAll": "explicit",
    "notebook.source.organizeImports": "explicit"
  }
}
```

DEFINE

Ruff can be **integrated** with various editors and IDEs

Install ruff extension for Visual Studio Code
Add configurations to **format**, **fix**, and **organize** imports on-save!

RUN

Ruff will **automatically execute** on save for Python or Jupyter Notebook files

breathe

HOW?



REPRODUCIBILITY



Ruff

CONVENTIONS



CONSISTENCY

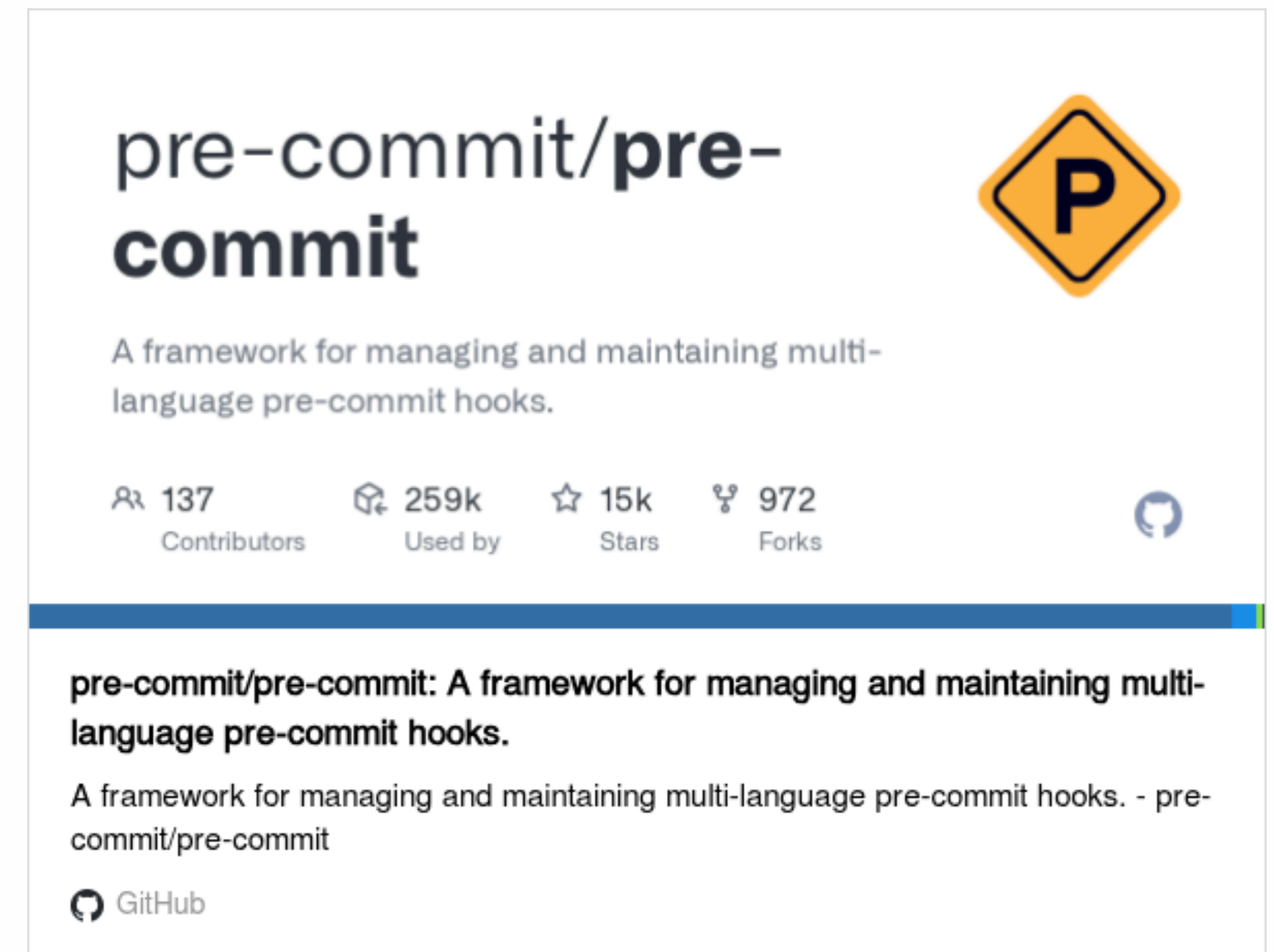


15.3K

Stars

"A framework for managing and maintaining multi-language **pre-commit hooks**."

"**Git hook scripts** are useful for identifying simple issues before submission to code review. By pointing these issues out before code review, this allows a code reviewer to focus on the architecture of a change while not wasting time with trivial style nitpicks."





7.8K

Stars

“prek is a reimagined version of pre-commit, built in **Rust**. It is designed to be a faster, dependency-free and drop-in alternative for it”

j178/prek

< A fast Git hook manager written in Rust, designed as a drop-in alternative to pre-commit, reimagined.

114

Contributors

2k

Used by

8k

Stars

217

Forks

j178/prek: ⚡ A fast Git hook manager written in Rust, designed as a drop-in alternative to pre-commit, reimagined.

⚡ A fast Git hook manager written in Rust, designed as a drop-in alternative to pre-commit, reimagined. - j178/prek

GitHub



INSTALL

OPTION 1

CLI

```
uv add --dev prek
```

pyproject.toml

```
[dependency-groups]
dev = [
    "prek>=0.4.3",
]
```

OPTION 2

pyproject.toml

```
[dependency-groups]
dev = [
    "prek",
]
```

CLI

```
uv sync --only-dev
```

creates/updates

uv.lock



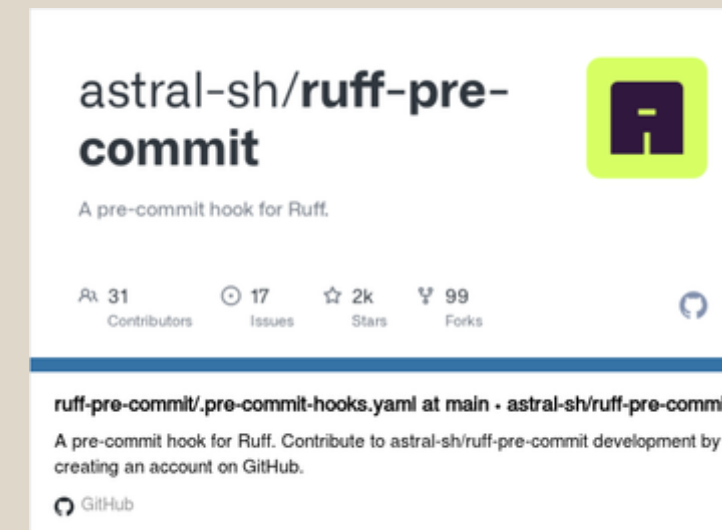
ADD CONFIGURATION

HOOK STRUCTURE

Add a *prek.toml* file with repository mapping to tell prek where to get the code for the hook from

```
[[repos]]
repo = "<link to a git package with a .pre-commit-hooks.yaml>"
rev = "<package version>"
hooks = [
  { id = "hook id found in .pre-commit-hooks.yaml",
    args = ["<param to pass to hook>"]
  },
]
```

HOOK FOR RUFF



prek.toml

```
[[repos]]
repo = "https://github.com/astral-sh/ruff-pre-commit"
rev = "v0.15.15"
hooks = [
  { id = "ruff-check",
    types_or = ["python", "pyi", "jupyter"],
    args = ["--fix"] },

  { id = "ruff-format",
    types_or = ["python", "pyi", "jupyter"]}
]
```




ADD CONFIGURATION

Add out-of-the-box hooks from pre-commit:

- Checks whether the files parse as valid python
- Ensures that a file is either empty, or ends with one newline
- Trim trailing whitespace
- checks yaml files for parseable syntax
- checks toml files for parseable syntax
- checks toml files for parseable syntax

prek.toml

```
[[repos]]
repo = "https://github.com/pre-commit/pre-commit-hooks"
rev = "v6.0.0"
hooks = [
    { id = "check-ast" },
    { id = "end-of-file-fixer" },
    { id = "trailing-whitespace" },
    { id = "check-yaml" },
    { id = "check-toml" },
    { id = "check-json" }
]
```



INSTALL HOOK SCRIPTS

RUN

Set up the git hook scripts locally

CLI

```
uv run prek install -f
```

Update hooks to latest version automatically
(bring hooks to latest tag on default branch)

CLI

```
uv run prek auto-update
```

Run hooks against all files when adding new
hooks (usually only run on changed files)

CLI

```
uv run prek run --all-files
```

creates

.git/hooks/pre-commit

```
#!/bin/sh
# File generated by prek: https://github.com/j178/prek
# ID: 182c10f181da4464a3eec51b83331688

HERE="$(cd "$(dirname "$0")" && pwd)"
PREK="/Users/gabrielharris/Documents/GitHub/DrGabrielHarris/pydata-norwich-sep-2025/.venv/bin/prek"

# Check if the full path to prek is executable,
otherwise fallback to PATH
if [ ! -x "$PREK" ]; then
    PREK="prek"
fi

exec "$PREK" hook-impl --hook-dir "$HERE" --script-version 4 --hook-type=pre-commit -- "$@"
```



RUN

Now prek will run automatically on git **commit**

```
git commit -m "Update docstring"
ruff check.....Failed
- hook id: ruff-check
- exit code: 1

D415 First line should end with a period, question mark, or exclamation point
--> src/main.py:16:5
    |
14 |     day_number: int,
15 | ) -> str:
16 |     """Formats day number with ordinal suffix"""
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
17 |     if 11 <= day_number % 100 <= 13:
18 |         suffix_text = "th"
   |
help: Add closing punctuation

Found 1 error.
No fixes available (1 hidden fix can be enabled with the `--unsafe-fixes` option).
ruff format.....Passed
check python ast.....Passed
fix end of files.....Passed
trim trailing whitespace.....Passed
check yaml.....(no files to check)Skipped
check toml.....(no files to check)Skipped
check json.....(no files to check)Skipped
uv-lock.....(no files to check)Skipped
uv-export.....(no files to check)Skipped
uv-export.....(no files to check)Skipped
```

Running hooks.

Hooks are either:

- Failed: violations > commit cancelled and fixes might be applied > new uncommented changes
- Passed: no violations
- Skipped: no files changes

JUST

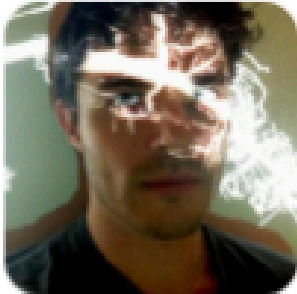
34.1K

Stars

“just is a handy way to save and run project-specific commands.”

casey/**just**

 Just a command runner



 215
Contributors

 258
Used by

 169
Discussions

 34k
Stars

 792
Forks



casey/just:  **Just a command runner**

 Just a command runner. Contribute to casey/just development by creating an account on GitHub.

 GitHub

JUST RECIPES

RECIPE STRUCUTRE

Add a *justfile* file with recipes

```
• • •  
@<name>: <prerequisites>  
    RECIPE
```

Run recipe in CLI

```
• • •  
just RECIPE
```

SETUP RECIPE

Add a *setup* recipe that has 2 prerequisites:

uv-sync recipe

prek-setup recipe

justfile

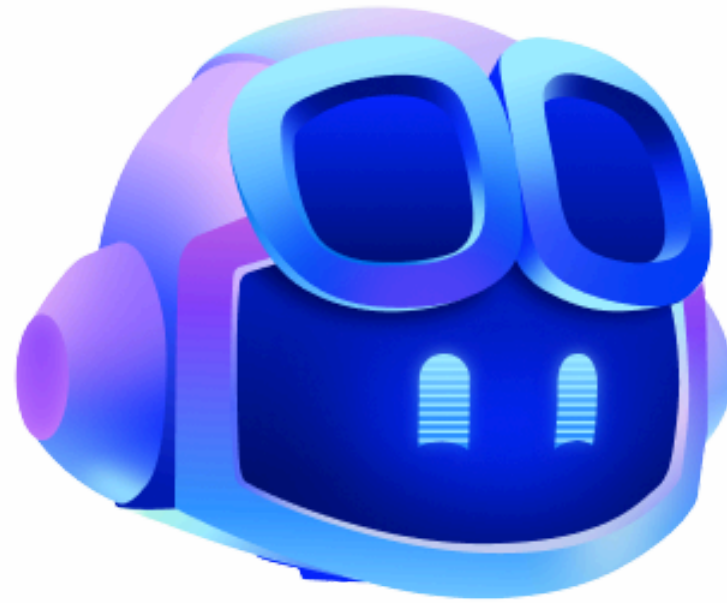
```
• • •  
set windows-shell := ["powershell.exe", "-NoLogo", "-Command"]  
  
@setup: uv-sync prek-setup  
    echo "Setting up..."  
  
@uv-sync:  
    uv sync --all-groups  
    uv export --all-groups --no-hashes --no-annotate --output-file requirements.txt  
    uv export --only-group inference --no-hashes --no-annotate --output-file requirements_inference.txt  
  
@prek-setup:  
    uv run prek install -f  
    uv run prek auto-update
```

Run recipe in CLI

```
• • •  
just setup  
Setting up...
```

CLI

GITHUB COPILOT



GITHUB COPILOT INSTRUCTIONS

.github/copilot-instructions.md

Always-on instructions:

Define project-wide rules

Use to add conventions the whole team follows, like code style, naming patterns, or preferred libraries.

```
---
name: 'Python Coding Standards'
description: 'Coding standards for Python files'
applyTo: '**/*.py'
---

# Python Coding Standards

## Python Guidelines

...

## Naming Conventions

...

## Style Guide

...

## Docstring Guide

...

## Logging

...

## Error Handling

...
```

GITHUB COPILOT INSTRUCTIONS

.github\instructions\commit-message.instructions.md

File-based instructions:

Define specific file patterns or task descriptions

Use when different parts of your codebase need different rules

```
---
name: 'Commits Conventions'
description: 'Conventions for commit messages'
applyTo: '**'
---

# Commits Conventions

## Rules of a Git commit message

- Separate subject from body with a blank line.
- Limit the subject line to 50 characters.
- Capitalize the subject line.
- Do not end the subject line with a period.
- Use the imperative mood in the subject line.
- The subject line should only summarize the change without why.
- Don't add a body.
```


GITHUB COPILOT AGENT SKILLS

.github/skills/just/SKILL.md

Create a skill that runs a script

Write instructions that tell
Copilot how to use the script

```
---
name: just
description: Run commands using the `just` tool.
---

When asked to run a just recipe, examine the `justfile` script at the
root directory to find the available recipes and their descriptions.
You can also use the `just -l` command to list available recipes.

Recipes include:

* Setting up the project.
* Syncing dependencies.
* Running all pre-commit hooks.
* Formatting all files in the current directory.
* Linting all files in the current directory.
```

join us!



The banner image features the PyData logo, which consists of three stacked cubes in blue, orange, and grey. To the right of the cubes, the text "PyData" is written in a large, sans-serif font, with "Py" in blue and "Data" in orange. Below this, the word "Norwich" is written in a smaller, white, sans-serif font. The background of the banner is a dark, grayscale photograph of a street in Norwich, showing traditional British architecture and a church spire in the distance.

PyData Norwich

PyData Norwich is a community of Python and Data Science professionals in Norwich, UK. The group has origins in the Python User Group, which was part of the NorDev community, and Data Science Norfolk. We host many events througho...

 Meetup / Jun 5